Memory Management in C

Stack vs. Heap, Dynamic Allocation Functions, Memory Leaks, and Buffer Overflows

Contents

1. Stack vs. Heap Memory
2. Allocation Functions (malloc, calloc, realloc, free)
3. Memory Leaks & Prevention
4. Buffer Overflows
5. Garbage Collection in C
6. Code Example: Dynamic Arrays

1. Stack vs. Heap Memory

Memory management in C refers to how a program allocates and frees memory blocks during execution. Because C provides low-level control, it is extremely flexible but requires careful handling to prevent crashes, memory leaks, and buffer overflows.

- **Stack Memory:** Used for local variables and function calls. It operates on a Last-In, First-Out (LIFO) principle and is automatically freed when functions terminate.
- **Heap Memory:** Used for dynamic memory allocation. Managed explicitly by the programmer during runtime. If not freed properly, it results in memory leaks.

2. Core Allocation Functions

Dynamic memory management relies on functions provided in the `<stdlib.h>` library:

- `malloc(size)`: Allocates a specified number of bytes and returns a pointer to the first byte (uninitialized values).
- `calloc(num, size)`: Allocates memory for an array and initializes all bytes to zero.
- `realloc(ptr, new_size)`: Resizes a previously allocated memory block (expands or shrinks).
- `free(ptr)`: Deallocates previously allocated memory to return it to the system.

3. Memory Leaks & Buffer Overflows

Memory Leak: Occurs when dynamically allocated memory is never released via `free()`, gradually exhausting system memory over time.

Buffer Overflow: Occurs when data exceeds the allocated boundaries of a buffer, corrupting adjacent memory regions and causing crashes or severe security vulnerabilities.

Note: C does not feature an automatic Garbage Collector; all resource lifecycles must be managed manually. Also, `<stdlib.h>` is required for dynamic allocation and random number generation functions.

4. Type Casting & Code Example: Dynamic Allocation & Pointer Arithmetic

CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    // Type Casting Example
    int x = 3;
    float y = 5.6;
    x = (int)y; // Explicit type casting from float to int

    // Dynamic array allocation for different data types
    int* numbers = (int*)malloc(sizeof(int) * 100);
    float* f_numbers = (float*)malloc(sizeof(float) * 100);
    char* chars = (char*)malloc(sizeof(char) * 100);

    // 1. Accessing elements using Subscript Operator
    for(int i = 0; i < 100; i++) {
        numbers[i] = rand() % 100;
    }

    // 2. Accessing elements using Pointer Arithmetic
    for(int i = 0; i < 100; i++) {
        *(numbers + i) = rand() % 100;
    }

    // Using calloc and realloc
    int* numbers2 = (int*)calloc(10, sizeof(int));
    numbers = (int*)realloc(numbers, sizeof(int) * 15);
    numbers = (int*)realloc(numbers, sizeof(int) * 5);

    // Freeing allocated memory to prevent memory leaks
    free(numbers);
    free(f_numbers);
    free(chars);
    free(numbers2);

    return 0;
}
```

SAMPLE RUN

```
Dynamic memory successfully allocated, populated via pointer arithmetic, resized, and
freed safely.
```